

INTERVIEW PREPARATION SERIES

Python Fundamentals

Interview Preparation

30 Questions on Structure, Variables, Data Types, I/O, Operators & Expressions

ABOUT THIS DOCUMENT

A conversational, interview-toned walkthrough of Python fundamentals — **30 questions across 6 core topics**, written for engineers with ~3 years of experience. Each answer pushes beyond the textbook definition into the practical knowledge that surfaces in real interviews and code reviews.

TOPIC 01

Structure of a Python Program

QUESTION 01

“Walk me through what actually happens between typing `python script.py` and your code running. I want the real pipeline, not ‘Python is interpreted’.”

ANSWER

“Python is interpreted” is a half-truth. CPython compiles your source to bytecode first, then a virtual machine executes the bytecode. Three phases.

Phase 1: lexing and parsing. The `script.py` source is tokenized, parsed into an AST. Syntax errors come from this phase.

Phase 2: compilation. The AST is compiled to bytecode — a sequence of instructions for the Python VM. You can see it: `python -m dis script.py`. The bytecode is cached in `__pycache__/script.cpython-312.pyc` so subsequent runs skip recompilation when the source hasn’t changed.

Phase 3: execution. The CPython VM (a stack-based interpreter written in C) walks the bytecode, instruction by instruction. Each opcode — `LOAD_FAST`, `BINARY_ADD`, `CALL_FUNCTION` — manipulates a value stack.

At 3 years you should also know that PyPy uses a tracing JIT (compiles hot loops to native code), and Python 3.13 introduced an experimental JIT into CPython itself. “Interpreted vs compiled” is a spectrum, and Python has been quietly moving along it for years.

QUESTION 02

“What’s the difference between a Python `module`, a `package`, and a `__init__.py` file?”

ANSWER

- **Module:** a single `.py` file. `import os` loads the `os` module.
- **Package:** a directory containing modules. `import requests.adapters` means “the `adapters` module inside the `requests` package.”
- **`__init__.py`:** historically, the marker that a directory is a package. Its contents run when the package is first imported — useful for re-exports, defining `__all__`, initialization.

Modern wrinkle: Python 3.3+ supports *namespace packages* — directories without `__init__.py` that can still be imported as packages, spread across multiple locations on the path. Useful for plugin systems, but if you don’t need that, an explicit (even empty) `__init__.py` is clearer.

Common use of `__init__.py`: flatten the API. Instead of `from mypkg.utils.string_helpers import upper`, you put `from .utils.string_helpers import upper` in `mypkg/__init__.py` so users can do `from mypkg import upper`. Library design move.

QUESTION 03

“Why does Python use indentation for blocks instead of braces or `begin/end`? What’s the real consequence?”

ANSWER

Indentation *is* the syntax. The parser uses `INDENT` and `DEDENT` tokens to recognize block boundaries. There’s no ambiguity about which block a statement belongs to — visual layout matches semantic structure.

Practical consequence: you can’t have lazily-formatted Python where the indentation lies. In C++ or Java, `if (x) foo(); bar();` looks like both are in the `if` but only one is. Python forbids the ambiguity at the syntax level.

Trap: mixing tabs and spaces. Python 3 forbids this entirely — `TabError` at parse time. Python 2 allowed it and produced subtle bugs. The PEP 8 rule is four spaces, never tabs. Configure your editor.

Trap 2: code-generation. If you're templating Python from another language, you must preserve indentation faithfully. Heredocs with leading whitespace are a frequent source of `IndentationError`.

QUESTION 04

“What does `if __name__ == '__main__':` actually do, and why is it there?”

ANSWER

Every module has a `__name__` attribute. When Python runs a file directly, `__name__` is set to `"__main__"`. When the same file is *imported* by another module, `__name__` is set to the module's dotted name (e.g., `"mypackage.utils"`).

The idiom `if __name__ == '__main__':` lets a file be both a runnable script and an importable module. Code under that guard runs only when executed directly — import doesn't trigger it.

```
# tool.py
def process(x):
    return x * 2

if __name__ == "__main__":
    import sys
    print(process(int(sys.argv[1])))

# Now: `python tool.py 5` prints 10
# And: `from tool import process` doesn't run the CLI parser
```

Without the guard, `import tool` would print `10` the first time the module loaded — because the CLI logic would run as a side effect of import. Confusing, and breaks tests that don't pass `sys.argv`.

QUESTION 05

“Explain the difference between `import x` and `from x import y`. Are there cases where one breaks and the other works?”

ANSWER

`import x` binds the name `x` in the current namespace to the module object. You access its contents as `x.something`.

`from x import y` binds the name `y` directly. The module `x` itself isn't accessible (unless you import it separately).

Cases where it matters:

- **Circular imports:** `from a import foo` in `b.py` requires `foo` to exist on `a` at the moment `b` imports. If `a` hasn't finished loading `foo` yet (because it's in the middle of importing `b`), you get `ImportError: cannot import name 'foo'`. `import a` delays the lookup; `a.foo` works as long as `a` finishes loading before you call it.
- **Monkey-patching:** if you `from logging import getLogger` and someone else monkey-patches `logging.getLogger`, your bound name still points at the original. `import logging; logging.getLogger()` sees the patched version.
- **Namespace pollution:** `from os import *` dumps everything public from `os` into your namespace. PEP 8 considers it harmful outside REPL use. Be explicit.

QUESTION 06

“Is Python an interpreted language? Defend your answer in detail.”

ANSWER

Not in the simple sense. CPython compiles to bytecode, then interprets the bytecode. That makes it more like Java (compile to JVM bytecode, run on JVM) than like, say, BASIC running line by line.

The *language* Python isn't interpreted or compiled — those are properties of *implementations*. CPython is mostly an interpreter (with caching). PyPy is a JIT compiler that produces native code at runtime. Cython compiles Python to C. Nuitka compiles to standalone executables. Same language, very different execution models.

Practical answer for an interviewer: “Python is typically interpreted via the CPython bytecode VM. The interpretation is what gives you dynamic typing, eval,

runtime introspection — but it's also why a CPU-bound loop in pure Python is 30–80× slower than the equivalent C. That's why production Python often delegates hot loops to NumPy, Cython, or native extensions.”

Don't over-claim. If someone asks specifically about “why is Python slow,” the answer is more about dynamic dispatch and reference-counting overhead than about “being interpreted.”

TOPIC 02

Variables

QUESTION 07

“In Python, when you write `x = 5`, what actually happens? It's different from C++ and Java.”

ANSWER

Python variables are names bound to objects, not storage locations holding values. `x = 5` creates (or reuses, since small integers are cached) an integer object `5`, then binds the name `x` in the current scope's namespace dictionary to point at it.

There's no “type of `x`” in the C sense — the type lives on the object, not on the variable. Reassign: `x = "hello"` — the name `x` now points to a different object. The integer `5` still exists if anything else references it; otherwise garbage collection eventually reclaims it.

Consequence: `a = b` doesn't copy `b`. It just binds `a` to the same object. For immutable types (`int`, `str`, `tuple`) this is invisible. For mutable types (`list`, `dict`), this is the source of “why did my list change when I modified the other one?” bugs.

The mental model: every Python variable holds a pointer. Assignment moves the pointer; it doesn't copy the pointee. Once that clicks, identity vs equality, mutability vs immutability, and default-argument gotchas all become consistent.

QUESTION 08

“What’s the difference between ``is`` and ``==``? When does the answer surprise you?”

ANSWER

`==` calls `__eq__` — “do these objects represent the same value?” `is` checks `id(a)`
`== id(b)` — “are these literally the same object in memory?”

```
a = [1, 2, 3]
b = [1, 2, 3]
a == b    # True (same contents)
a is b    # False (different list objects)

a = b     # now a and b point to the same object
a is b    # True
```

Surprises:

- Small integers are cached. `a = 256; b = 256; a is b` is True. `a = 257; b = 257; a is b` is False (or True depending on context). The `- 5 to 256` range is interned in CPython. Never rely on it.
- String interning. `"foo" is "foo"` is usually True; `"f" + "oo" is "foo"` might or might not be. Implementation detail.
- Use `is` only for `None`, `True`, `False`, and sentinel objects. `if x is None` is the idiomatic null check — it works even when `__eq__` is overridden to do something weird (e.g., NumPy arrays return elementwise comparisons).

QUESTION 09

“Describe Python’s scoping rules. What’s LEGB?”

ANSWER

Python resolves names in four scopes, in order: **L**ocal, **E**nclosing, **G**lobal, **B**uilt-in. The first match wins.

- **Local**: names defined in the current function body.
- **Enclosing**: names in any enclosing function’s scope (closures). Multiple levels possible.
- **Global**: names at the module level (top of file).
- **Built-in**: names from `builtins` — `print`, `len`, `range`, etc.

Assignment inside a function makes the name local by default. To assign to an outer scope, you need keywords: `global x` for module level, `nonlocal x` for enclosing function scope (closures).

```
x = 1
def outer():
    x = 2
    def inner():
        nonlocal x    # without this, the next line creates a local
        x = 3
    inner()
    print(x)    # 3
outer()
print(x)       # 1
```

Common surprise: `UnboundLocalError`. You *read* a variable, then later assign to it — Python sees the assignment, marks the variable local for the whole function, then the read at the top fails because the local hasn't been assigned yet.

QUESTION 10

“What does this print, and why?”

```
def append_to(item, lst=[]):
    lst.append(item)
    return lst

print(append_to(1))
print(append_to(2))
print(append_to(3))
```

ANSWER

It prints `[1]`, `[1, 2]`, `[1, 2, 3]` — not three separate `[1]`, `[2]`, `[3]` lists like most people expect.

Why: default arguments are evaluated *once* — at function-definition time, not at each call. The empty list `[]` is created when `def append_to` runs. That single list becomes the default. Every call that doesn't pass an explicit list mutates the *same* list.

This is the most famous Python gotcha. It applies to any mutable default: lists, dicts, sets, custom objects.

Fix: use `None` as the sentinel.

```
def append_to(item, lst=None):
    if lst is None:
        lst = []
    lst.append(item)
    return lst
```

At 3 years you should *never* write `def f(x=[])` or `def f(x={})`. Linters flag this. Reviewers reject this.

QUESTION 11

“Explain `global` and `nonlocal`. When have you genuinely needed each?”

ANSWER

`global x` declares that `x` inside the function refers to the module-level `x`. Without it, `x = 5` inside a function creates a local.

`nonlocal x` (Python 3+) does the same for the nearest enclosing function’s scope — not global. Used in closures that need to *write* to the captured variable.

```
def make_counter():
    count = 0
    def increment():
        nonlocal count
        count += 1
        return count
    return increment

c = make_counter()
c(); c(); c() # returns 3
```

Without `nonlocal`, `count += 1` would create a local `count` shadowing the closure, and you’d get `UnboundLocalError` because you’re trying to read it before assigning.

When you’d need `global`: rarely. Most cases where you think you need it are better handled by passing arguments or using a class. The exceptions: lazy initialization of expensive module-level resources, plugin registration patterns, debugging.

When you'd need `nonlocal`: closure-based counters, decorators that track state, callback patterns. Often a class with `__call__` is cleaner.

TOPIC 03

Data Types

QUESTION 12

“Name Python’s built-in container types and tell me when you’d choose each.”

ANSWER

- `list`: ordered, mutable, allows duplicates. Default sequence. $O(1)$ random access, $O(n)$ insert in the middle, $O(1)$ amortized append.
- `tuple`: ordered, *immutable*. Use for fixed-shape records (returning multiple values from a function, dict keys, set members).
- `dict`: hash map. $O(1)$ average lookup/insert. Since Python 3.7, insertion order is preserved. Use for key-value mappings, caches, configuration.
- `set`: hash set. $O(1)$ membership test. Use for deduplication, set algebra (intersection, union, difference).
- `frozenset`: immutable set. Hashable, so it can be a dict key or set member.
- `str`, `bytes`, `bytearray`: text and binary sequences. `str` is Unicode, immutable. `bytes` is raw bytes, immutable. `bytearray` is mutable bytes — useful for parsing or building binary protocols.

From `collections`: `deque` ($O(1)$ append/pop from both ends), `Counter`, `defaultdict`, `OrderedDict` (now redundant with dict order guarantee in 3.7+, kept for backwards compat and explicit signaling).

QUESTION 13

“What’s the difference between a list and a tuple, beyond ‘one is mutable’? Be specific.”

ANSWER

- Memory: tuples are smaller and faster to construct. CPython optimizes tuple creation aggressively — a tuple literal in source is often a precomputed constant.

- Hashability: tuples of hashable elements are themselves hashable. Lists never are. So `dict[(1, 2)]` works; `dict[[1, 2]]` raises `TypeError`.
- Semantic intent: a tuple typically means “a fixed-shape record” — `(name, email, age)`. A list typically means “a homogeneous sequence of varying length.” Tuples for heterogeneous bundles, lists for sequences of similar things.
- Identity behavior: empty tuples are interned (`() is ()` is `True`). Empty lists are not (`[] is []` is `False`). Implementation detail you should never depend on, but illustrates the optimization gap.

Modern Python: prefer `@dataclass` or `NamedTuple` for records of more than 2–3 fields. Bare tuples lose readability fast — `user[2]` is much worse than `user.age`.

QUESTION 14

“Why are strings immutable in Python? What do you gain, and what do you lose?”

ANSWER

Gain: hashability (so strings can be dict keys), interning (identical literals share storage), thread safety (no synchronization needed for read-only data), and security (a method that validates a string can’t have it mutated between validation and use).

Lose: string-building gets expensive. `s = ""; for x in items: s += str(x)` reallocates and copies the entire string on each iteration — $O(n^2)$ total work.

The Pythonic fix is `"".join(str(x) for x in items)` — single allocation, $O(n)$. For more complex assembly, `io.StringIO` or formatted strings (`f"{prefix}{body}{suffix}"`).

Worth noting: CPython actually has an optimization for `s += x` in some cases (it can resize in place if `s` has only one reference). But you can’t rely on it across implementations or Python versions. Write the idiomatic version.

QUESTION 15

“Why is `0.1 + 0.2 != 0.3` in Python? Show me you understand floating-point, not just ‘it’s a known bug’.”

ANSWER

It’s not a Python bug — it’s IEEE 754 double-precision floating-point doing what it’s specified to do. Python’s `float` is a thin wrapper around C’s `double`.

In binary, `0.1` is an infinite repeating fraction — the same way $1/3$ is in decimal. The 64-bit closest approximation is `0.1000000000000000055511...`. Same for `0.2`. Add them, you don't get exactly `0.3` — you get `0.30000000000000004`. The `==0.3` check sees `0.3`'s closest approximation, which is a different rounded value.

How to handle: never use `==` on floats. Use `math.isclose(a, b)` — it has sensible default tolerances and a `rel_tol/abs_tol` pair. For money, use `decimal.Decimal`.

```
from decimal import Decimal
Decimal("0.1") + Decimal("0.2") == Decimal("0.3")    # True
```

Watch the constructor: `Decimal(0.1)` gives you the inexact double representation. `Decimal("0.1")` gives you exactly 0.1. Always construct from strings.

QUESTION 16

“What’s the difference between ``==`` on a ``dict`` and ``==`` on a ``set``? Both are hash-based.”

ANSWER

Both are content equality, order-independent.

- `dict_a == dict_b`: equal if they have the same keys mapped to equal values.
- `set_a == set_b`: equal if they have the same elements.

Both work in $O(\min(\text{len}(a), \text{len}(b)))$ on average — they iterate one and look up in the other. They don't depend on insertion order, even though dict preserves it now.

Where it gets interesting: hash-based equality compares by `__eq__` within hash buckets. If your custom class has a broken `__eq__`/`__hash__` combo, dict and set behave unpredictably. The contract: equal objects must hash to the same value.

Performance trap: comparing two large dicts where values are themselves big nested structures can be very expensive. The `==` recurses through value comparisons.

QUESTION 17

“When would you use a ``dataclass`` versus a regular class versus a ``NamedTuple``?”

ANSWER

- **@dataclass** (Python 3.7+): a class with auto-generated `__init__`, `__repr__`, `__eq__` from your field declarations. Mutable by default; `frozen=True` makes it immutable. Use for plain data carriers that might gain a few helper methods.
- **NamedTuple**: immutable, tuple-based, with field names. Like a tuple you can read by name. Use when immutability and tuple-compatible iteration matter, e.g., for keys, public APIs, or interop with code expecting tuples.
- **Regular class**: when behavior dominates data. If the class has more methods than fields, or invariants that need enforcing in setters, write the class explicitly.

Modern advice for new code: start with `@dataclass`. It's the cleanest. Move to a hand-written class when you outgrow it. Reach for `NamedTuple` only when you specifically need tuple semantics.

Alternative ecosystem: Pydantic (`BaseModel`) for runtime validation, `attrs` for the original library that inspired dataclasses (more powerful, more dependencies). For most cases, the stdlib `@dataclass` is plenty.

TOPIC 04

Input / Output

QUESTION 18

“Why do we use ``with open(...)`` instead of just ``open(...)``?”

ANSWER

The `with` statement (context manager) guarantees cleanup. When the block exits — normally, by return, or by exception — `__exit__` runs and closes the file.

```
with open("config.json") as f:
    data = json.load(f)
# f is closed here, even if json.load raised
```

Without `with: f = open(...)` then `f.close()` later. If anything between them throws, `close` never runs. The file descriptor leaks. On long-running services or scripts that open many files, you eventually hit “too many open files” errors.

CPython *often* closes the file when `f` is garbage-collected (reference count hits zero) — but you can’t rely on it. PyPy doesn’t use refcounting; finalization is non-deterministic. Even in CPython, if `f` is captured by a closure, exception traceback, or other reference, GC is delayed.

Universal rule: any resource with explicit lifetime (files, sockets, locks, database connections) belongs in a `with` block. If a library you’re using doesn’t support context manager protocol, wrap it with `contextlib.closing` or write a `@contextmanager` function.

QUESTION 19

“What’s the difference between `print(x)` and `sys.stdout.write(str(x) + '\n')`? Are they equivalent?”

ANSWER

Not quite equivalent. `print` is a built-in function with conveniences:

- Calls `str()` on each argument.
- Joins multiple args with a separator (default `' '`, overridable via `sep=`).
- Adds an `end` character (default `'\n'`, overridable via `end=`).
- Targets `file=` (default `sys.stdout`) — you can redirect to `stderr` or any file-like object.
- Has `flush=` — default `False`, so output is buffered.

`sys.stdout.write` is a thin wrapper around the underlying file handle. No `str()` conversion, no separator handling, no newline. Faster (no overhead), but you handle everything yourself.

Use `print` for normal output. Reach for `sys.stdout.write` when you need raw control — progress bars that overwrite themselves with `\r`, custom output framing, or measurable performance optimization in tight loops.

QUESTION 20

“You read a 50 GB log file with `open(path).read()`. What goes wrong, and how do you do it right?”

ANSWER

`read()` without arguments reads the whole file into memory as a single string. On a 50 GB file with 32 GB of RAM, you get `MemoryError` — or, worse, your OS starts swapping and grinds to a halt.

Right way: stream. Files are iterable; iteration yields one line at a time, buffered.

```
with open(path) as f:
    for line in f:
        process(line)
```

Constant memory regardless of file size. The underlying read buffer is 8 KB by default; you can tune it via `open(path, buffering=65536)` if you need to.

For binary or unknown-structure data: `f.read(8192)` in a loop, processing chunks. For huge files where you need random access, `mmap.mmap` lets the OS page in just the parts you touch. Avoid loading the whole thing.

Common gotcha: `for line in f.readlines()` looks similar to `for line in f`, but `readlines()` first loads everything into a list. Same memory disaster as `read()`. The iterator protocol on the file object is what makes streaming work.

QUESTION 21

“You’re reading user input with `input()`. What’s the type of what you get back, and what are the gotchas?”

ANSWER

`input()` always returns a string — even if the user types 42. You’re responsible for type conversion. `int(input("age: "))` parses an integer.

Gotchas:

- No automatic conversion. `input() + 1` raises `TypeError` — string plus int.
- Trailing whitespace usually isn’t included — the newline that ended the input is stripped. But leading spaces are preserved if the user typed them. `.strip()` is the defensive move.
- Conversion errors raise. `int("hello")` raises `ValueError`. For user input, wrap in `try/except` and prompt again, or use `input()` with explicit validation.

- On end-of-file (Ctrl-D on Unix, Ctrl-Z on Windows), `input()` raises `EEOFError`. Scripts running with redirected input need to handle this.

For anything beyond toy programs, consider `argparse` (CLI arguments), `click` (richer CLI), or `getpass` (passwords — doesn't echo). `input()` is fine for tutorials and `__main__` demos.

TOPIC 05

Operators

QUESTION 22

“What’s the difference between `/` and `//` in Python? And what changed between Python 2 and 3?”

ANSWER

`/` is true division — always returns `float`. `7 / 2 == 3.5`, `4 / 2 == 2.0` (float, not int).

`//` is floor division — returns the floor of the true result. `7 // 2 == 3`, `-7 // 2 == -4` (floor, not truncate). Type follows operand types: `int // int = int`; `float // float = float`.

Python 2 change: `/` did integer division when both operands were ints. `7 / 2 == 3` in Python 2; `7 / 2 == 3.5` in Python 3. Python 3 split the operators to be explicit — you must say what you want.

Watch the floor direction. `-7 // 2` is `-4`, not `-3`. That’s floor toward negative infinity, *different* from C-style truncation. If you specifically want truncation, `int(a / b)` — but be aware of floating-point error on edge cases.

QUESTION 23

“Walk me through operator precedence with `and` and `or`. What does `1 and 2 or 3` evaluate to, and why?”

ANSWER

`and` and `or` don’t return booleans — they return one of their operands.

- `a and b`: returns `a` if `a` is falsy, else `b`. Short-circuits.
- `a or b`: returns `a` if `a` is truthy, else `b`. Short-circuits.

1 and 2 or 3: 1 and 2 first. 1 is truthy, returns 2. Then 2 or 3: 2 is truthy, returns 2. Final result: 2.

Idiomatic uses: `name = user_name or "anonymous"` — default fallback when `user_name` is falsy (empty string, None). `safe = obj and obj.value` — guard against None.

Subtle bug: `x = some_dict.get("key") or default` returns `default` if the value is falsy — not just missing. If `0`, `""`, or `False` are valid values, use `x = some_dict.get("key", default)` instead. The `or` pattern conflates “missing” with “falsy.”

QUESTION 24

“What does Python’s `*` operator do? Cover the cases.”

ANSWER

Many things depending on context:

- **Multiplication:** `3 * 4 == 12`. Numeric types.
- **Sequence repetition:** `[0] * 5` creates `[0, 0, 0, 0, 0]`. `"ab" * 3` gives `"ababab"`. Trap: `[[0]] * 3` creates three references to the *same* inner list — modifying one modifies all.
- **Argument unpacking:** `f(*args)` unpacks an iterable into positional arguments. `f(**kwargs)` unpacks a dict into keyword arguments.
- **Argument collection:** `def f(*args)` collects positional args into a tuple. `def f(**kwargs)` collects keyword args into a dict.
- **Keyword-only marker:** `def f(a, b, *, c)` — the bare `*` means “every parameter after this is keyword-only.”
- **Iterable unpacking in literals:** `[*a, *b]` concatenates iterables into a list. `{**d1, **d2}` merges dicts.

And in 3.5+: `**` in dict literals, `*` in tuple/list literals. Same operator, six distinct grammatical roles. Context is everything.

QUESTION 25

“Explain `is` vs `==` once more, but in the context of comparing to None. What’s the right idiom?”

ANSWER

Always `is None`, never `== None`. Two reasons.

Reason 1: `None` is a singleton. There's exactly one `None` object in a running Python program. Identity (`is`) is exactly the right check.

Reason 2: `==` calls `__eq__` on the left operand. Custom classes can override `__eq__` in surprising ways. `x == None` where `x` is a NumPy array returns an array of booleans, not a scalar — and using it as a boolean condition raises an error.

```
# Right
if value is None:
    ...

# Wrong (works for plain values, breaks on NumPy/Pandas)
if value == None:
    ...
```

Same logic applies to `True` and `False`, but more often you just want truthiness: `if value:`, not `if value == True:`. The latter forces a boolean comparison and misses the truthy-but-not-True cases (non-empty lists, non-zero numbers, etc.).

TOPIC 06

Expressions

QUESTION 26

“What’s a list comprehension, and when is it the right tool versus a generator expression or a for loop?”

ANSWER

A list comprehension builds a list eagerly: `[x*2 for x in items if x > 0]`. Reads top-to-bottom as “for each `x` in `items` where `x > 0`, give me `x*2`.” Result is a fully-materialized list.

A generator expression looks similar but with parens: `(x*2 for x in items if x > 0)`. Produces values lazily — one at a time, computed on demand. No list ever exists in memory.

Choose by need:

- Need a list (multiple passes, indexing, `len()`) — list comprehension.
- Single pass, large or infinite source, downstream consumes one at a time (sum, max, any, for-loop) — generator expression. Constant memory.

- Side effects, complex control flow, multiple outputs — explicit for loop. Comprehensions should be expressions, not statements.

Nesting: `[x for row in matrix for x in row]` flattens. Order matches nested for-loops left-to-right. Past two levels of nesting, prefer a function or an intermediate variable — nested comprehensions stop being readable fast.

Performance: comprehensions are slightly faster than equivalent for-loops with `append` — the comprehension form is optimized in the bytecode. The win is usually 10–20%, not life-changing.

QUESTION 27

“What’s the difference between a generator function and a regular function that returns a list?”

ANSWER

A function with `yield` anywhere in its body is a generator function. Calling it doesn’t run the body — it returns a generator object. Each call to `next()` (or each iteration in a for loop) runs the body until the next `yield`, returns that value, and pauses. State is preserved between yields.

```
def squares_up_to(n):
    for i in range(n):
        yield i * i

# Calling doesn't run anything yet
g = squares_up_to(1000000)
# Now we consume
for s in g:
    if s > 100:
        break
    process(s)
```

Compare with returning a list: the list is fully built before the caller sees anything. 1 million squares allocated, then handed over. With the generator, only the squares you actually consumed exist — you stopped at `> 100`, so 11 values were computed.

When to use generators:

- Large or unbounded sequences — reading log files, network streams, pagination.

- Pipelines where each stage transforms the previous — generators chain efficiently.
- Early termination is likely.

Trap: generators can only be iterated once. `list(g); list(g)` gives `[...] []` — the second is empty. If you need to iterate twice, materialize to a list first or rebuild the generator.

QUESTION 28

“What’s the walrus operator `:=`, and where does it earn its keep?”

ANSWER

Python 3.8+. `:=` is an assignment expression — it assigns *and* returns the assigned value. Lets you assign inside expressions where statements aren’t allowed.

```
# Old idiom
while True:
    line = f.readline()
    if not line:
        break
    process(line)

# With walrus
while line := f.readline():
    process(line)
```

Real wins:

- Read-and-test loops (as above) — collapses three lines to one.
- Avoid duplicate computation: `if (n := len(items)) > 10: print(f"{n} is too many")` — the `len` call happens once.
- List comprehension with shared computation: `[y for x in data if (y := process(x)) is not None]`.

Overuse warning: walrus can hide what would be clearer as two lines. `if (result := compute()) and result.is_valid():` packs assignment, truth test, and method call into one expression — harder to read than the three-line version. Use it when the alternative is duplicate work or three-line ceremony, not just because it’s shorter.

QUESTION 29

“What’s truthiness in Python? How does `if x:` decide whether to enter the block?”

ANSWER

Every Python object is either truthy or falsy. `if x:` calls `bool(x)` implicitly. `bool(x)` calls `__bool__` if defined, falling back to `__len__` (zero length = falsy), defaulting to truthy if neither is defined.

Default falsy values:

- `False`, `None`
- Zero of any numeric type: `0`, `0.0`, `0j`, `Decimal(0)`, `Fraction(0, 1)`
- Empty sequences and collections: `''`, `[]`, `()`, `{}`, `set()`, `range(0)`

Everything else is truthy by default — including `"False"` (non-empty string), `[0]` (non-empty list with a falsy element), and objects without `__bool__` or `__len__`.

Idiomatic: `if items:` — enters the block when `items` is non-empty. Don’t write `if len(items) > 0:` — verbose, less Pythonic, sometimes slower.

Subtle trap: `if value:` can’t distinguish `None` from `0` from empty string. If the difference matters, `if value is None:` is the explicit check.

QUESTION 30

“You see `result = func() if condition else fallback()`. What’s special about this expression compared to a regular if statement?”

ANSWER

It’s a conditional expression (Python’s ternary). Returns `func()` if condition is truthy, else `fallback()`. Crucially: only one of the two is evaluated.

Differences from an `if` statement:

- It’s an expression — has a value, can be assigned, passed as argument, used inside a comprehension.
- Reads condition-in-middle, unlike most languages’ `condition ? a : b`. Pythonic but a one-time mental adjustment for newcomers from C/Java/JS.
- Only one branch evaluates — useful when the unused branch would error: `x = lookup(key) if key in cache else None` wouldn’t blow up on a missing key, because the lookup is skipped.

When to use: simple value selection. `display_name = user.name if user.name else "anonymous"` — readable. `x = compute() if heavy_condition() and`

`other_condition() else complex_fallback(arg)` — has crossed into unreadable. Break it up into a statement.

Watch out for nesting: `a if x else b if y else c` is `a if x else (b if y else c)` — nested ternary, right-associative. Past one level of nesting, write the statement.